FRP 2026 AI ENGINEERING TRACK DAY 2

How does AI work inside?

And how do we work with AI agents?

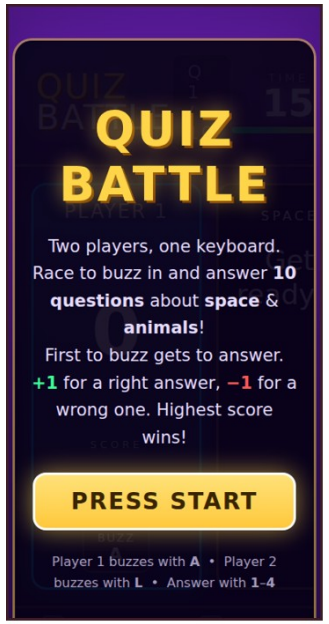
Vladimir Albrekht

ISSAI, Nazarbayev University, 25 October 2026

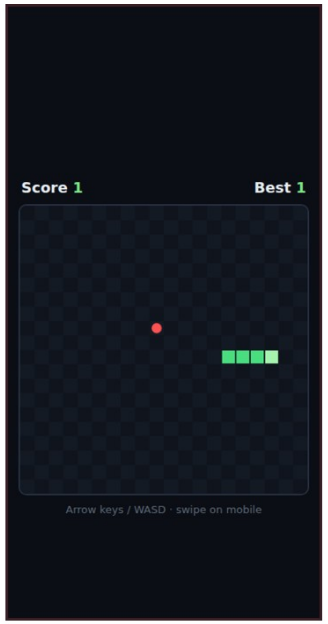
You type one sentence.
Five minutes later, you can play it.

What would you build?

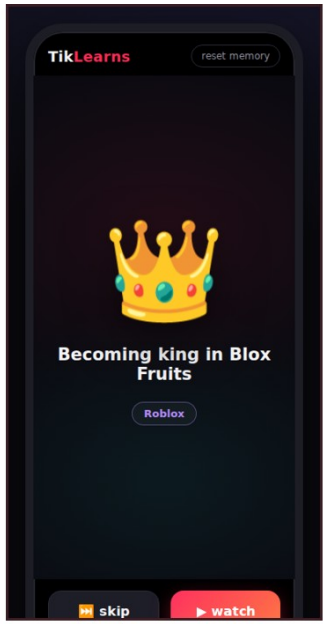
Our AI built these from one sentence



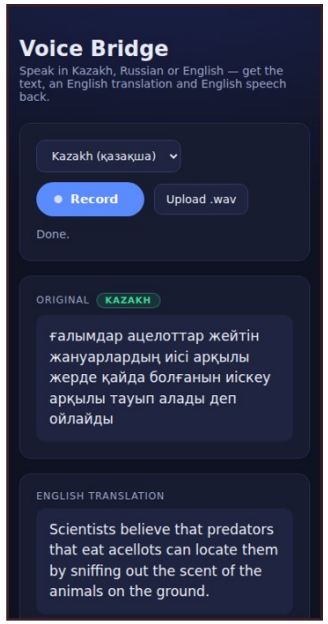
Quiz battle



Snake



Cat feed



Voice translator

Our first model (Qwen3.8) + the OpenCode agent, 12 Oct 2026. One sentence each, no help. This week: DeepSeek V4.1 Flash.

Today: how this AI works, and how you build with it.

START

LIVE

Watch: an agent builds a feed

The task I give the agent

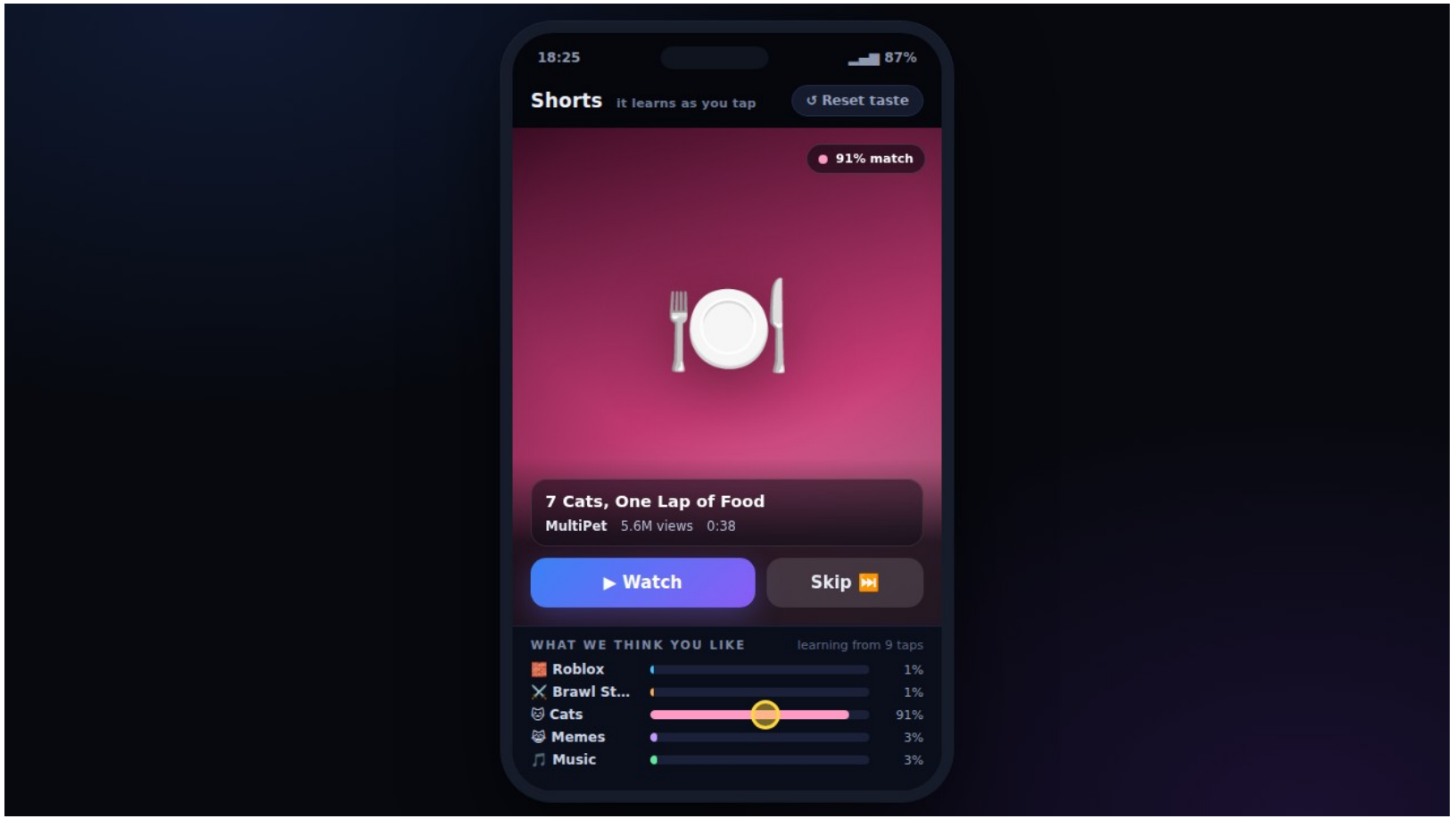
Make a phone page with a feed of short video cards about Roblox, Brawl Stars, cats, memes and music. Tap “watch” or “skip”. The page learns what I like and shows more of it, with bars for what it thinks I like. One HTML file (a web page).

By Saturday, you will build much bigger things this way.

START

DEMO

The agent builds the feed



Today, 4 parts

1. How it works

The model guesses the next piece of text

2. Where it fails

Short memory, and it can invent facts

3. AI agents

A model that uses tools in a loop

4. Your plan

Start task 1 of your week plan

PART A: HOW THE MODEL WORKS

The model guesses the next piece

One small skill, learned from a very large amount of text.

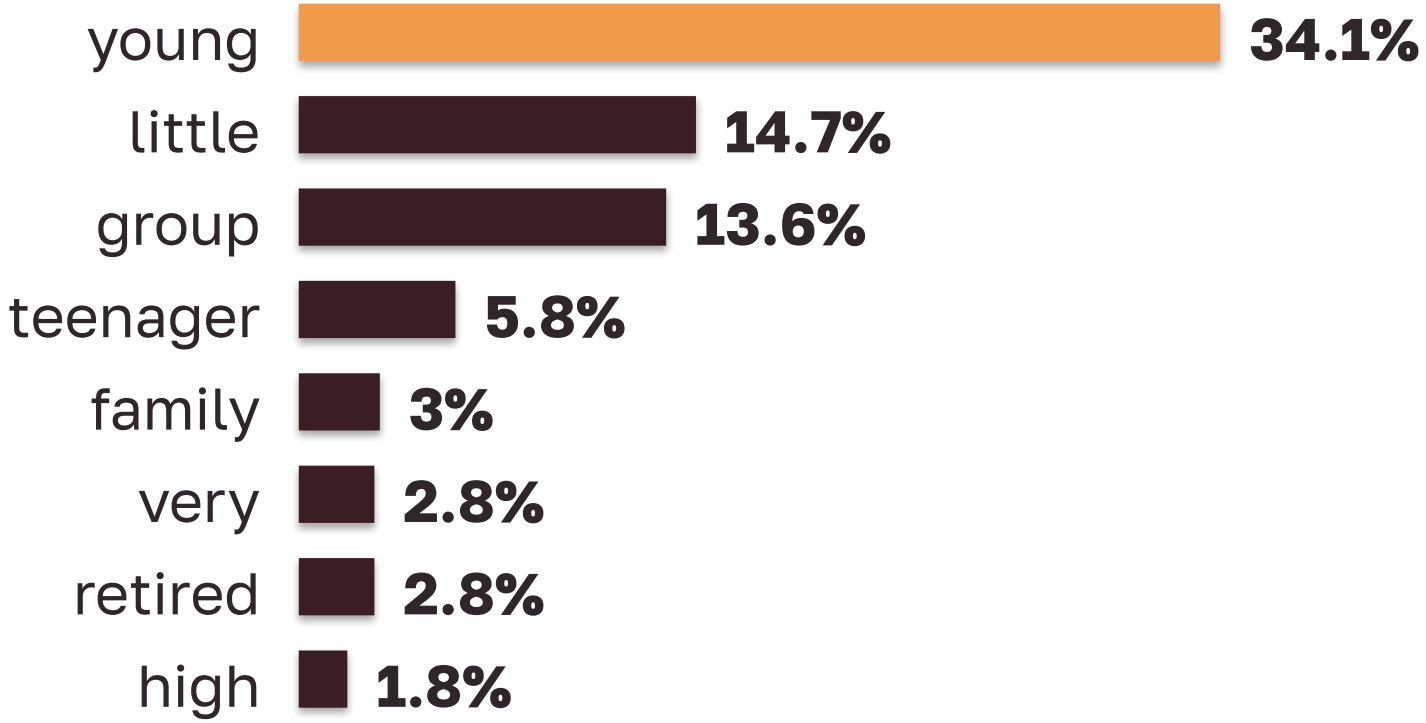
ACTIVITY, 3 MINUTES

Be the model

Once upon a time, there was a _____

Write the next word on paper. Only one word.

What our model guesses



Top 8 of thousands

The model gives every word a chance, like tickets in a lucky draw.

Qwen3.8 on our ISSAI server, 12 Oct 2026

The model does one thing: it guesses the next piece of text.

A long answer = many small guesses

Story 1

Once upon a time, there was **a teenager named Tom.**

Story 2

Once upon a time, there was **a family in a small town.**

Story 3

Once upon a time, there was **a little girl named Lily.**

NEW WORD

temperature = how random the draw is

Same start, 3 stories: each word is a new lucky draw.

The model reads in pieces: tokens

EN — 13

I | study | artificial | intelligence | at | Nazar | bay | ev | University | in | Ast | ana | .

RU — 21

Я | изуч | а | ю | искус | ственный | ин | тел | лект | в | Наз | ар | ба | ев | Универ | сит | ете | в | А | стане | .

KZ — 27

М | ен | А | стана | да | ғы | Наз | ар | ба | ев | Универ | сит | ет | інде | ж | ас | анд | ы | ин | тел | лект | ті | о | қ | им | ын | .

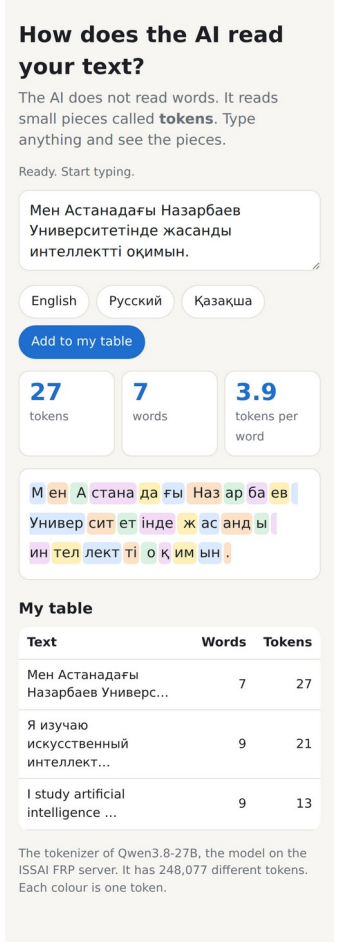
Why? It learned from little Kazakh text, so it knows fewer whole Kazakh words.

> **The same sentence: 13 pieces in English, 27 in Kazakh.**

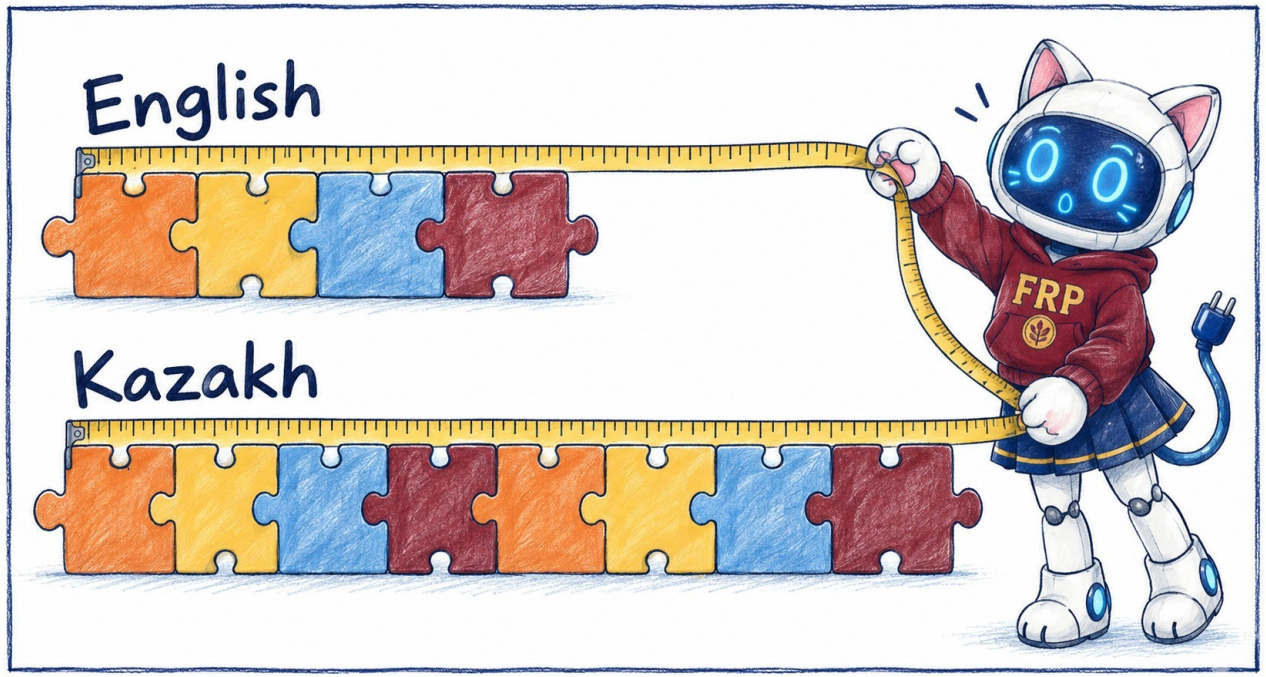
TOGETHER ON THE BIG SCREEN, 5 MINUTES

How does AI read your text?

- Your name, in any language
- One sentence in English, Russian and Kazakh
- Who can find a word with the most pieces?



Kazakh needs 2× more pieces



2.1×

more tokens for Kazakh, in 20 everyday sentences English 187, Russian 223, Kazakh 387 tokens. Slower and costs more.

This is one reason ISSAI trains models on more Kazakh text (KAZ-LLM).

Where did it learn this? By reading



15 trillion

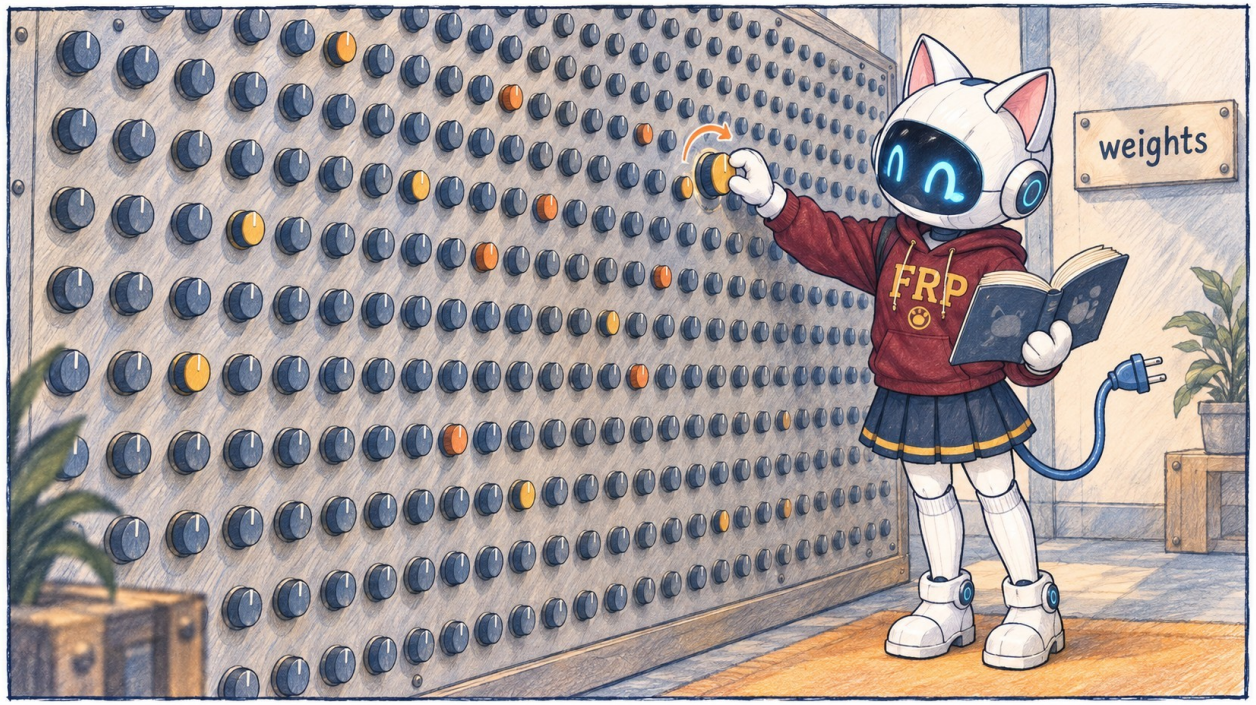
tokens read by Llama 3.1

85,000

years of reading for one person

> **No person could ever read this much. The model did it once, in training.**

Inside: billions of knobs



763 billion

knobs (weights) in our model

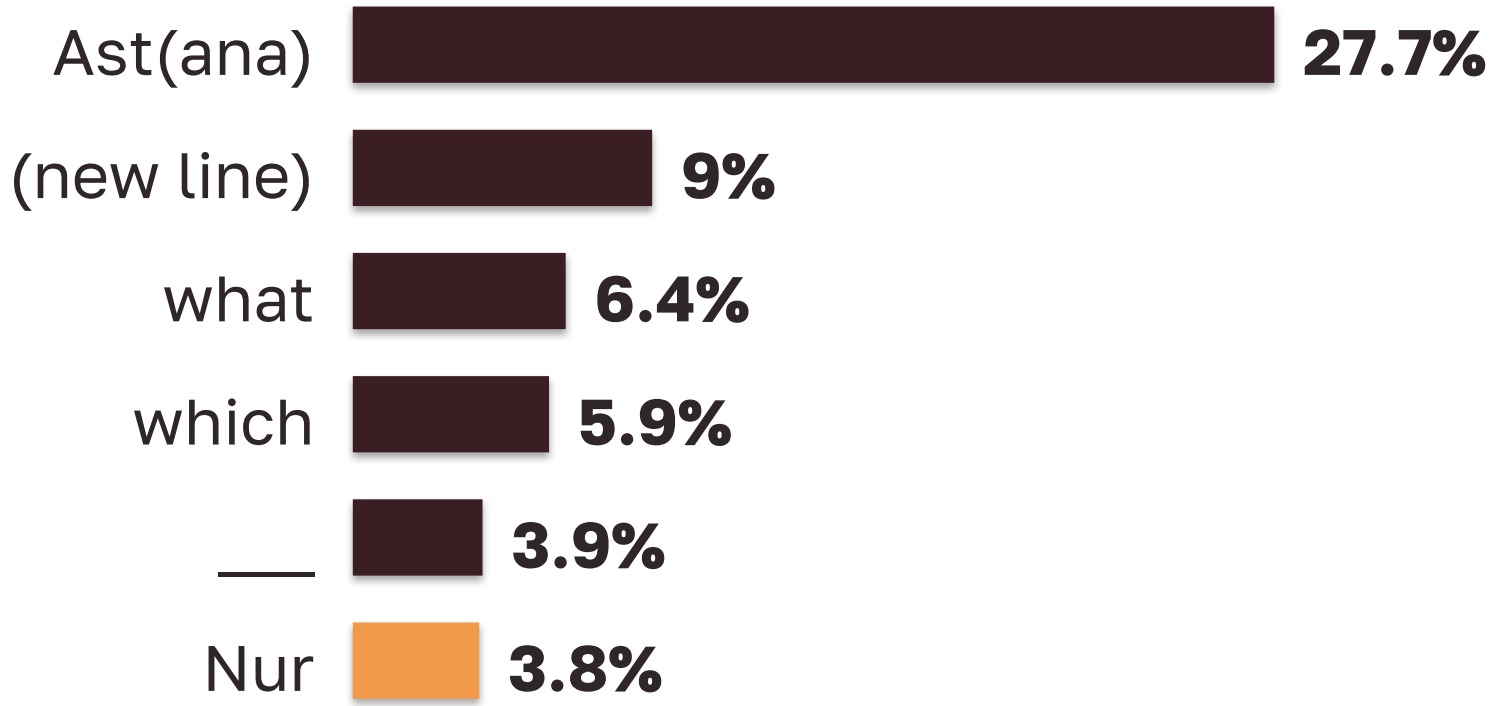
Like volume knobs. Wrong guess? Turn them a tiny bit, trillions of times.

NEW WORD

weights = learned numbers

Training turns the knobs. Nobody writes the rules by hand.

Nazarbayev University is in the city of …



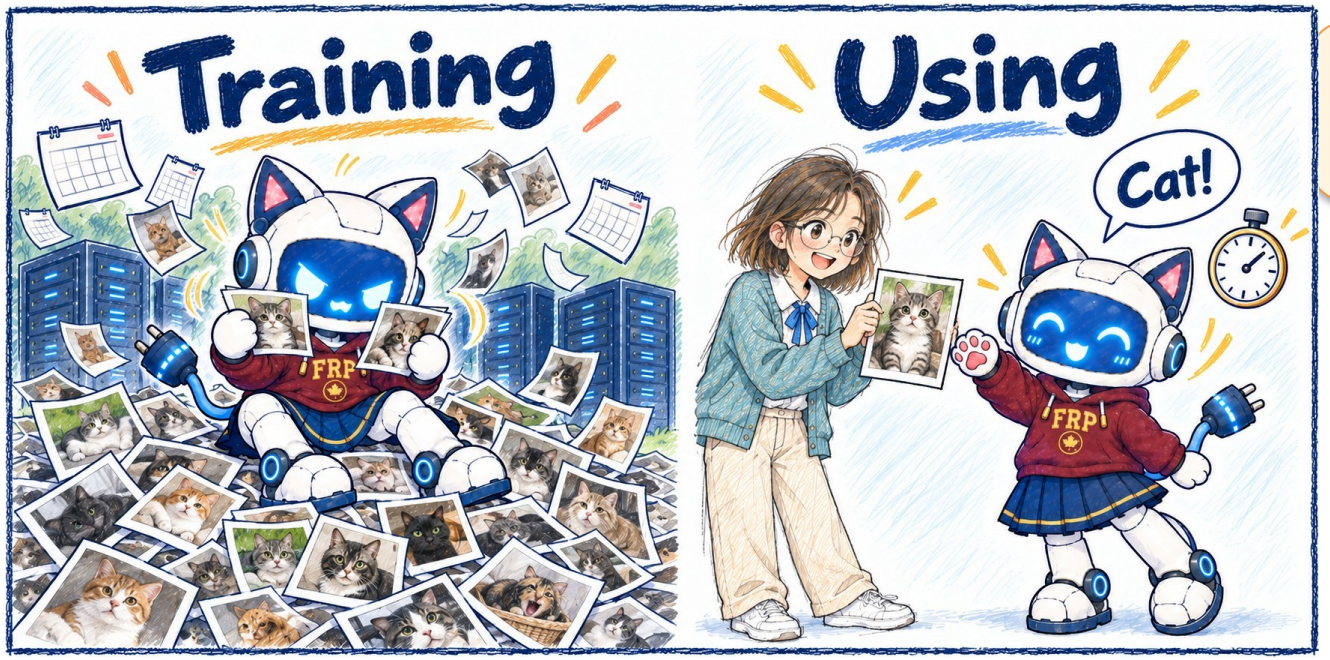
Astana wins.

But why is "Nur" here at all?

Astana was named Nur-Sultan from 2019 to 2022.

It learned from old text, so its knowledge can be old.

Training: years. Using: seconds.



NEW WORD

GPU = a graphics card

Training Llama 3.1 405B on one GPU: about 3,500 years. So they used thousands of GPUs.

You will use a trained model. You will not train one from zero this week.

HANDS UP, 5 MINUTES

Quiz time

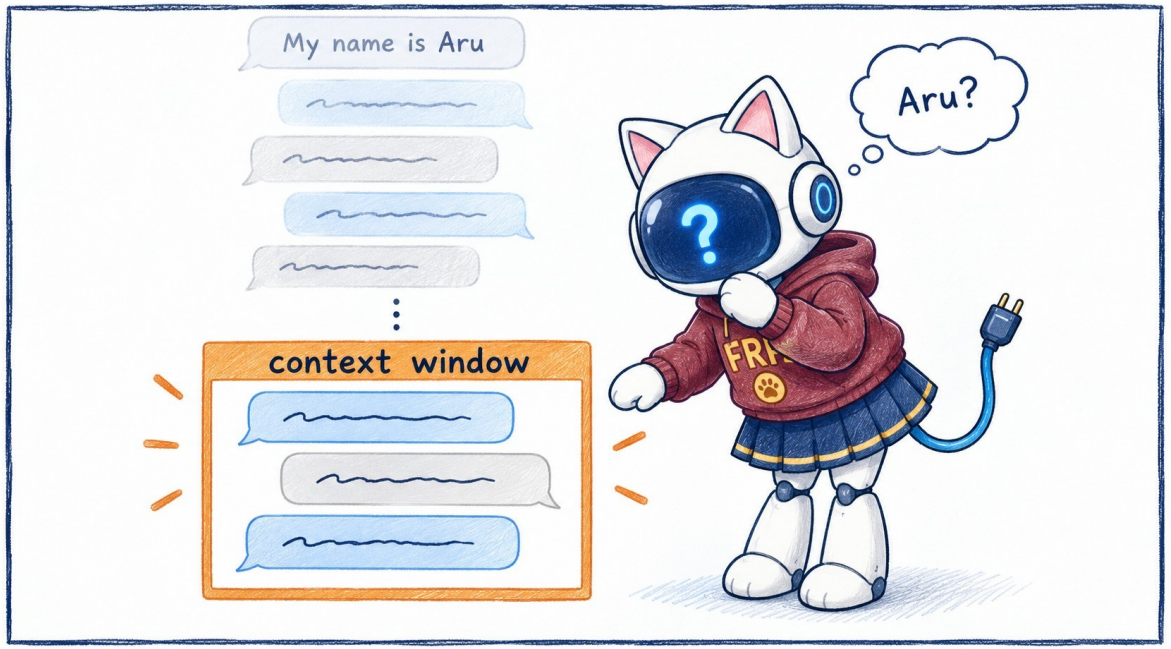
- What does the model do?
- Kazakh or English: which needs more tokens? Why?
- How long would one person need to read what the model read?
- Which takes years: training or using?
- **Bonus:** why can the same question give a different answer?

PART B: WHERE IT FAILS

What the model cannot do

It forgets, and it can invent facts. You must check.

AI memory: the context window



131,072

tokens the model sees at one time

About 330 book pages. In Kazakh: about 160.

The model only sees what is in its window.

Which city is this?



Photo: Dauren Nabijan (public domain)

We asked our AI model the same question.

What do you think it said?

Our first model: 4 wrong cities



Tries 1 to 4: our first model (Qwen3.8, 12 Oct).

A big model (GPT-6 Astra), 3 tries:

Astana, Astana, Astana

It was wrong and sure. Always check, even big models.

Our model now: right city, invented reasons



DeepSeek V4.1 Flash, 4 tries:

Astana, Astana, Astana, Astana

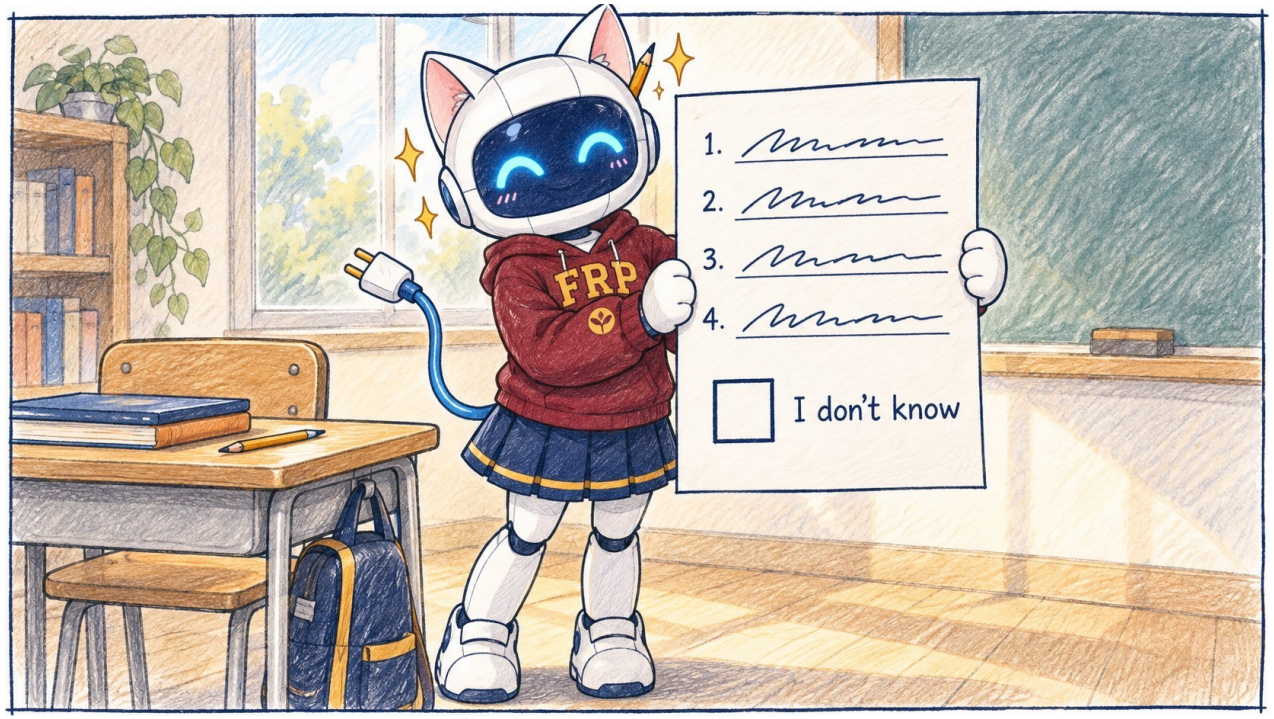
Try 1, the reason: “the golden-tipped **Bayterek** and **Khan Shatyr**”

Neither is in the photo.

Every try named different buildings.

A right answer can have an invented reason. Check the reasons too.

Why? It never leaves the answer blank



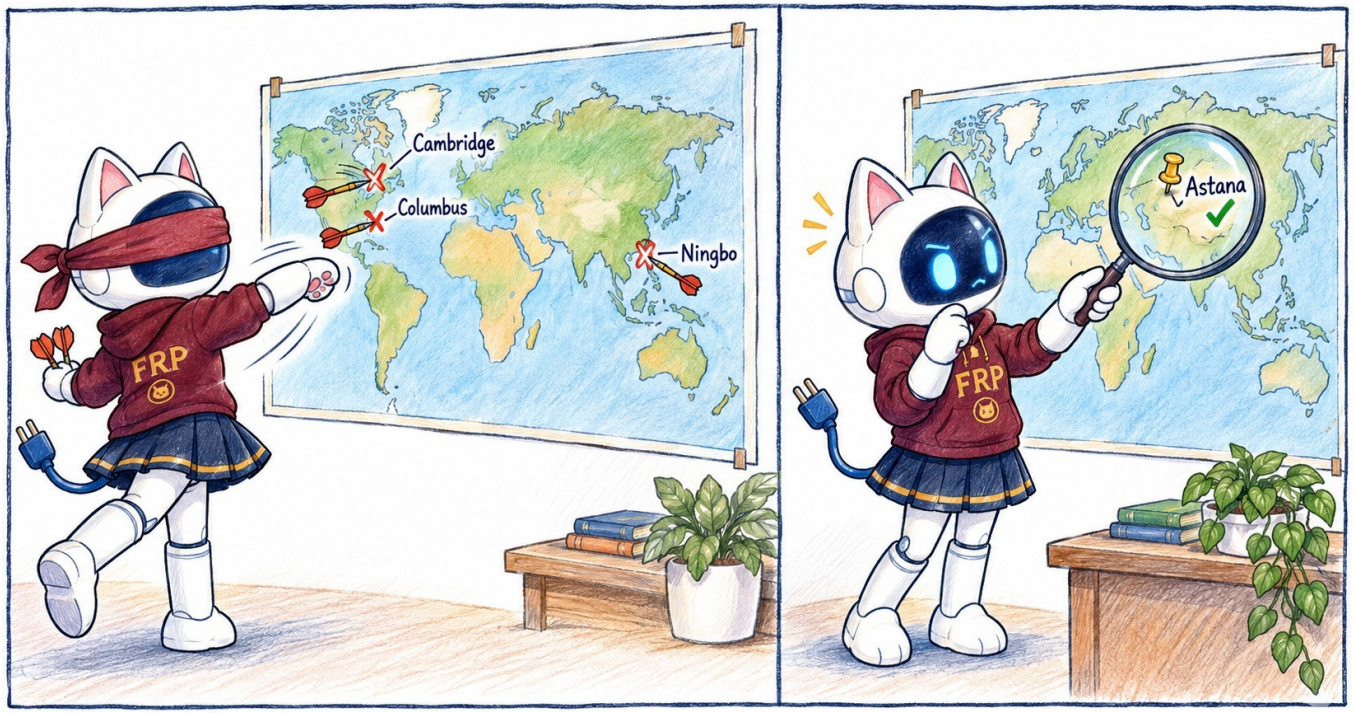
Like a student who never leaves an answer blank.

Training and tests reward a guess more than “I don’t know”.

“Why Language Models Hallucinate”, 2025

It sounds sure even when it is wrong. You must check.

What if the model could look it up?



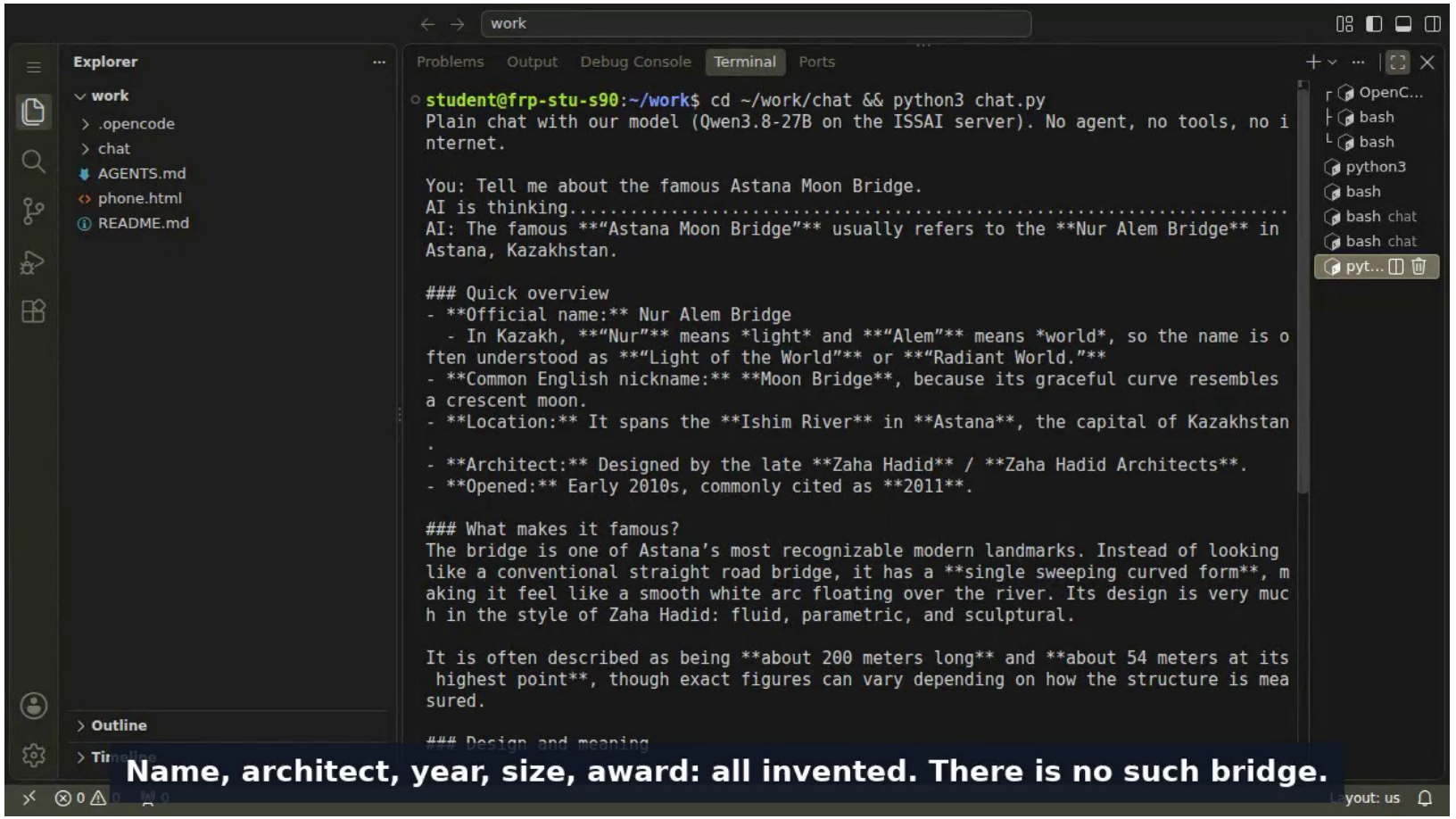
Tools let the model check. That is what AI agents do.

Let’s make the AI invent a fact

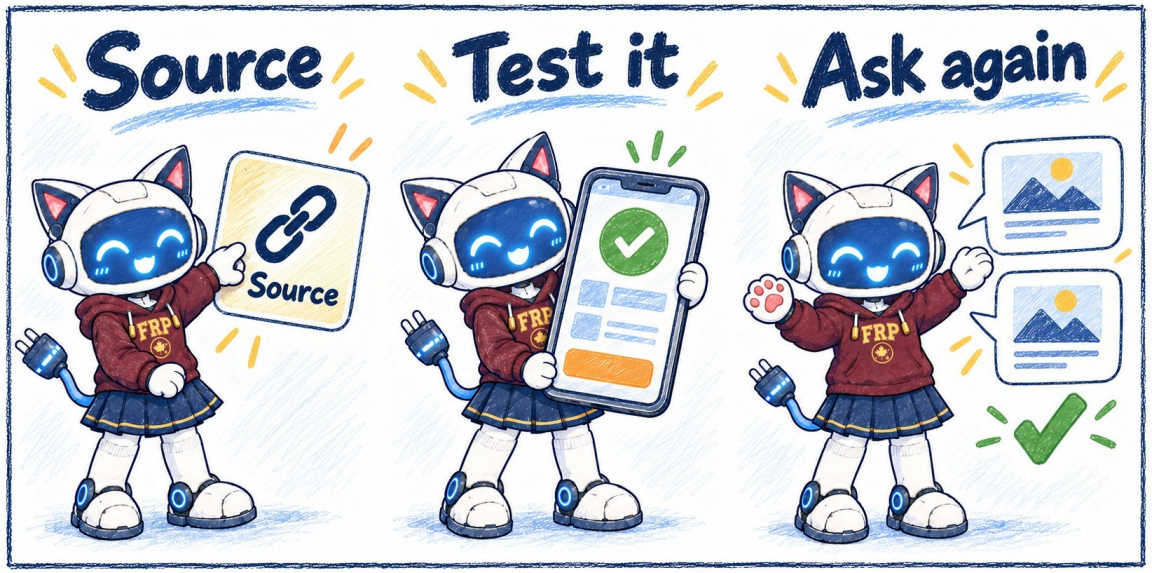
- You suggest a person, a place or a book that does not exist
- Example: “Tell me about the famous Astana Moon Bridge.”
- I ask our AI. Does it say “I don’t know”, or does it invent?

DEMO

Our AI invents a fact



3 ways to check



Open the link. AI can invent links too!

Run the code, open the page

Same answer twice? Not proof. Check a 2nd source

Trust, but check. Every time.

Break, 10 minutes

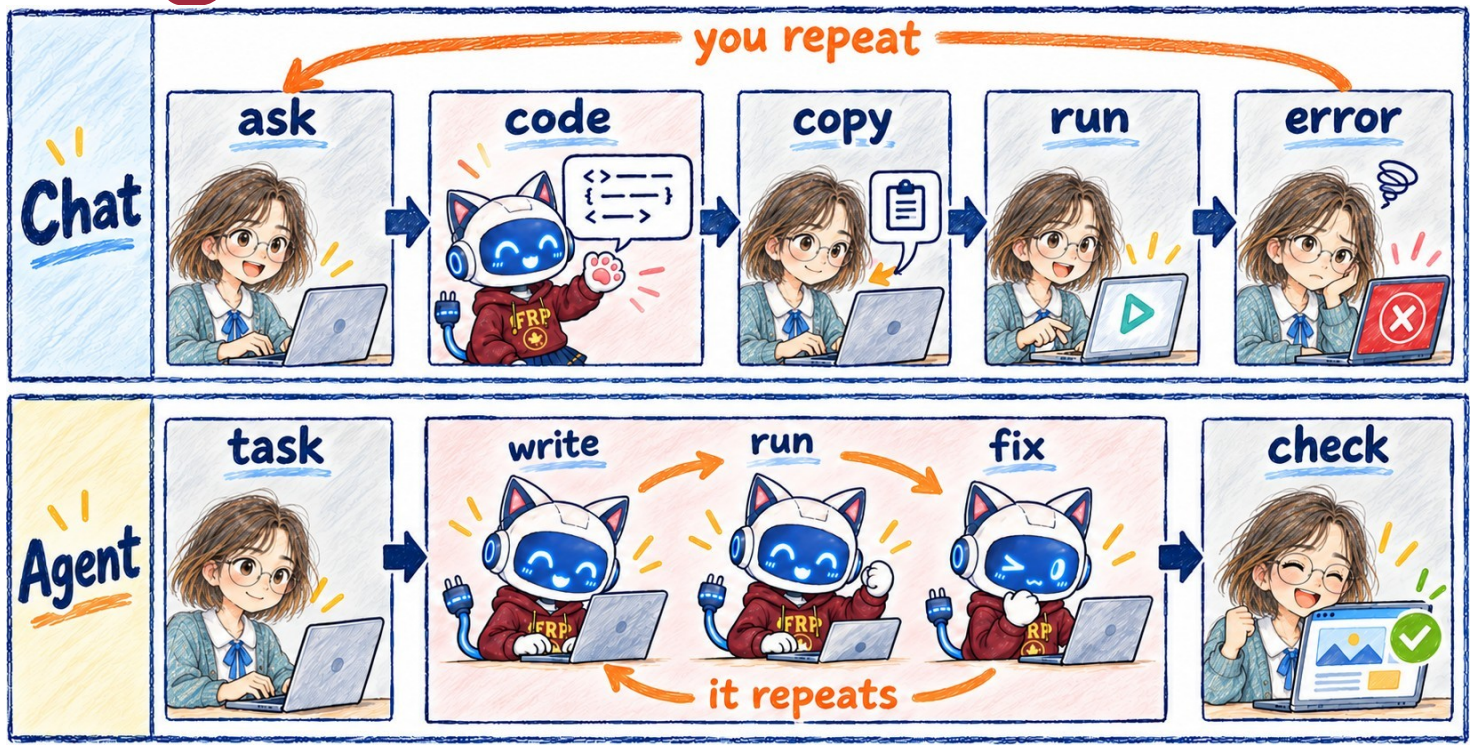
Next: AI agents

PART C: AI AGENTS

A model that uses tools in a loop

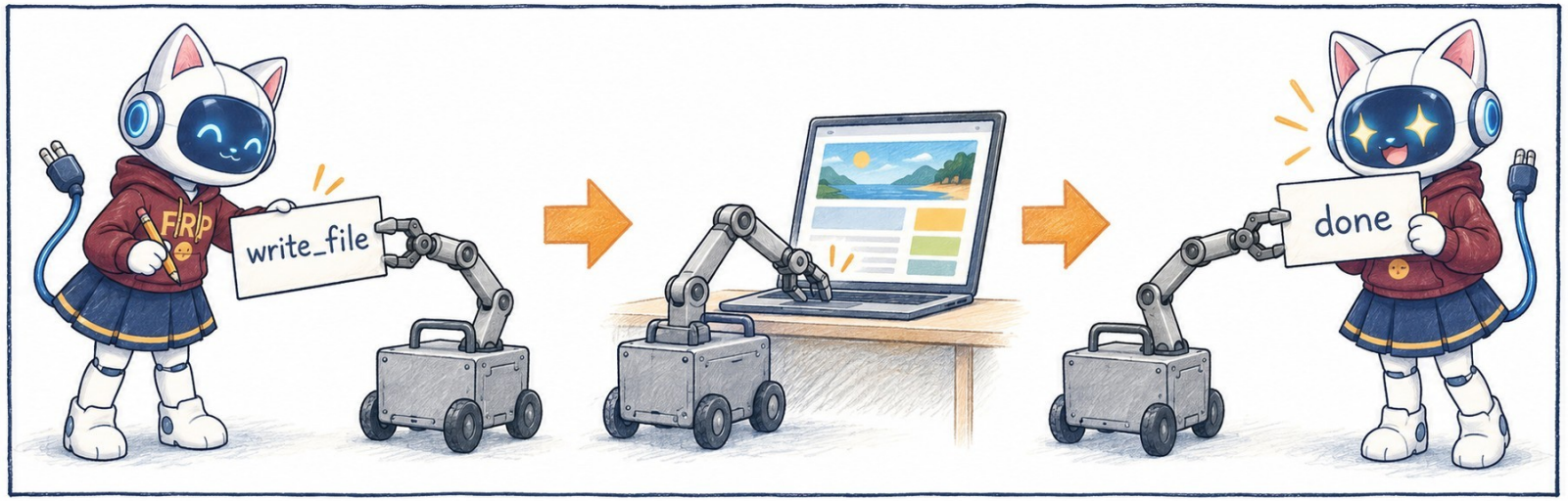
Think, act, look, repeat.

Chat vs agent



With a chat, you do the work. With an agent, it does the work and you check it.

The model only writes text. So how does it act?



1 It writes a note

“write the file”

2 A program does it

the file appears

3 Text comes back

“done”

Tools turn the model’s text into real actions.

A real tool call from our model

IN PLAIN WORDS

tool: write a file

file: index.html

text: the page

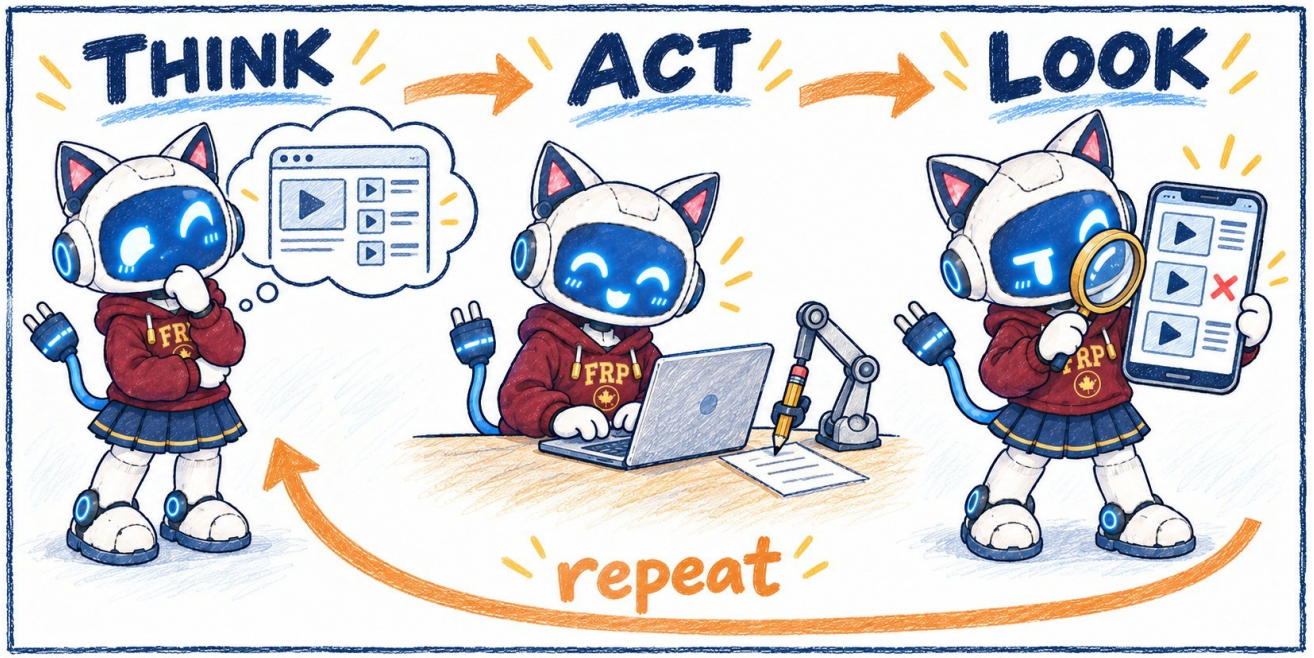
Our first model (Qwen3.8), 12 Oct 2026. Shortened.
Each model has its own format; the idea is the same.

WHAT THE MODEL REALLY WROTE

```
<tool_call>
<function=write_file>
<parameter=path>
index.html
</parameter>
<parameter=content>
<!DOCTYPE html> ... <title>My cat feed</title> ...
<h1>Cats</h1> ...
</parameter>
</function>
</tool_call>
```

> **That text is all the model writes. The agent program does the rest.**

The agent loop: think, act, look



Example: the watch button does nothing. Fix it, look again: it works.

But LOOK can only check what your task says.

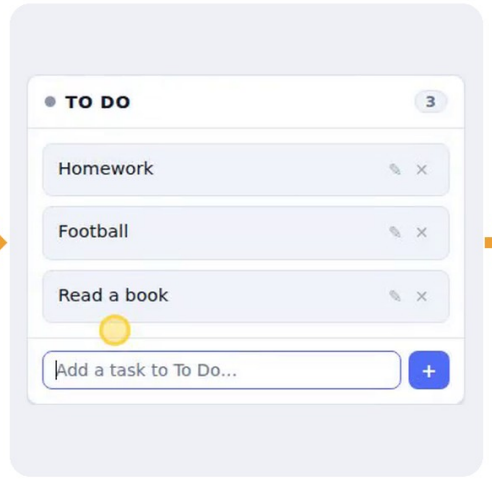
DEMO

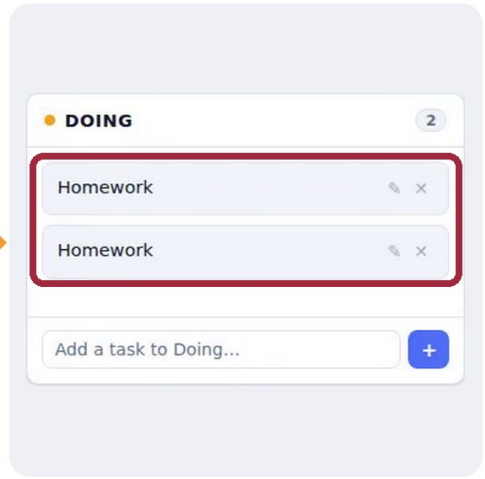
A real agent run: think, act, look

The agent's own note:

```
[DONE] Implement
index.html with all
required features
(columns, add, edit,
delete, DnD, ...)
```

DnD = drag and drop

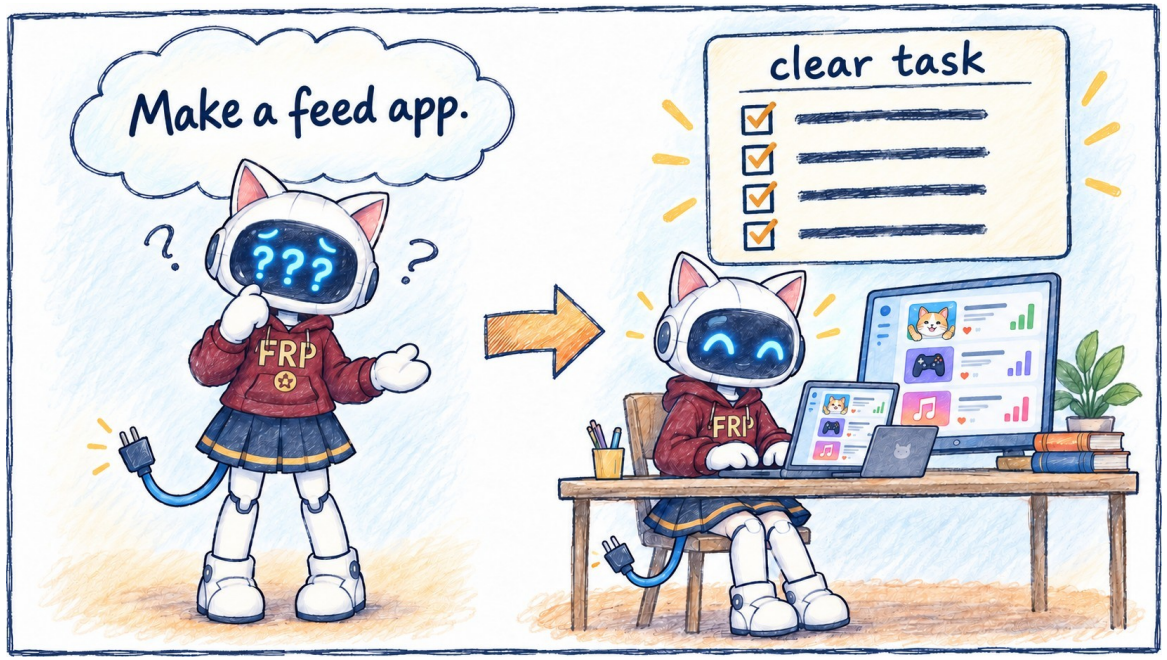




Agent: DONE ✓ **We drag one card** **Two cards ✗**

A real run from our tests, 12 Oct 2026. Click to play the recording.

Remember the feed? Why did it work?



Vague: "Make a feed app."

NEW WORD

vague = not clear, says too little

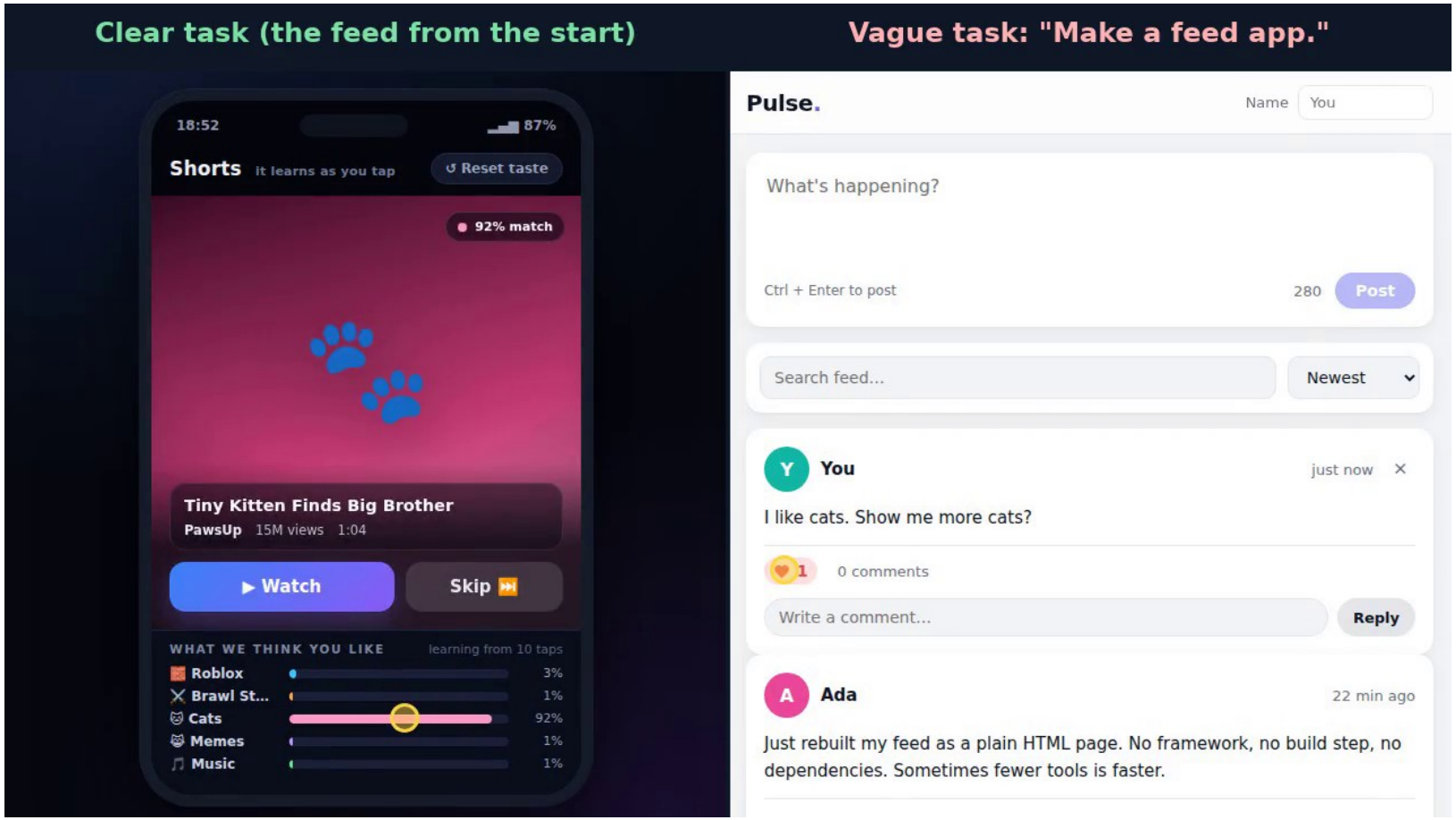
Clear: our task

"A phone feed. Tap watch or skip. It learns what I like."

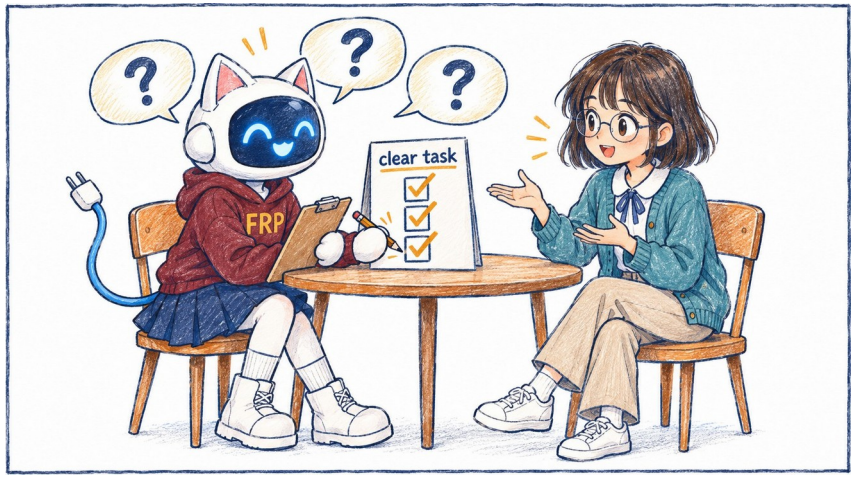
The agent is as good as your task.

DEMO

Vague task: “Make a feed app.”



Let the agent ask you first



in your workspace

/ask

Type: “Before you build, ask me up to 5 questions.”

You and the agent agree on the task before any code.

What our model asked, for real

It asked me

1. What will the feed show: text, images, videos?
2. Who will use it?
3. What is it for?
4. Web, iPhone or Android?
5. Any features you must have?

After my answers, it wrote

Goal: a phone feed page, no login

Input: watch and skip on 5 kinds of cards

Check: the bars change, liked cards come first

Limits: one HTML file

Its questions turned my vague idea into a clear task.

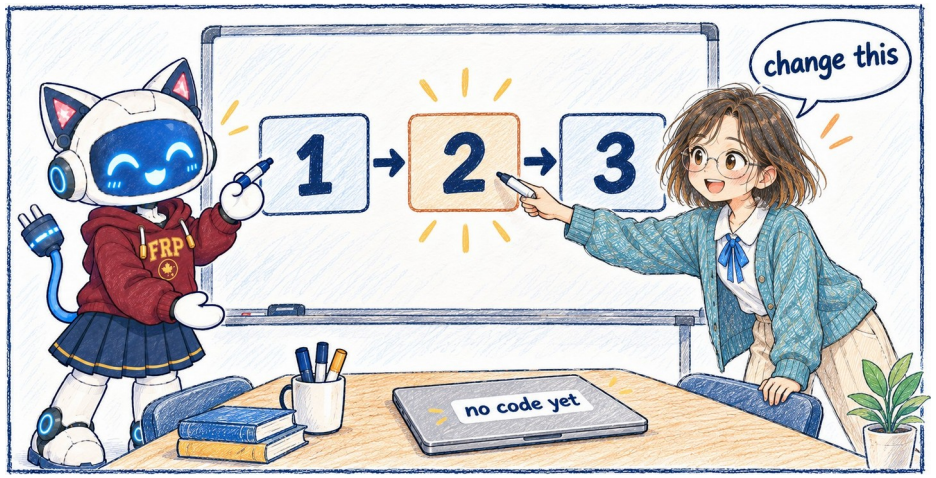
A clear task = 4 parts

Goal	Input	How to check	Limits
A phone feed that learns what I like	Cards: games, cats, memes, music	Tap watch on 3 cat cards: more cats?	One HTML file, no login

In a few minutes you use this recipe in your own workspace.

> **You are the engineer. The agent is your helper.**

Ask for a plan first



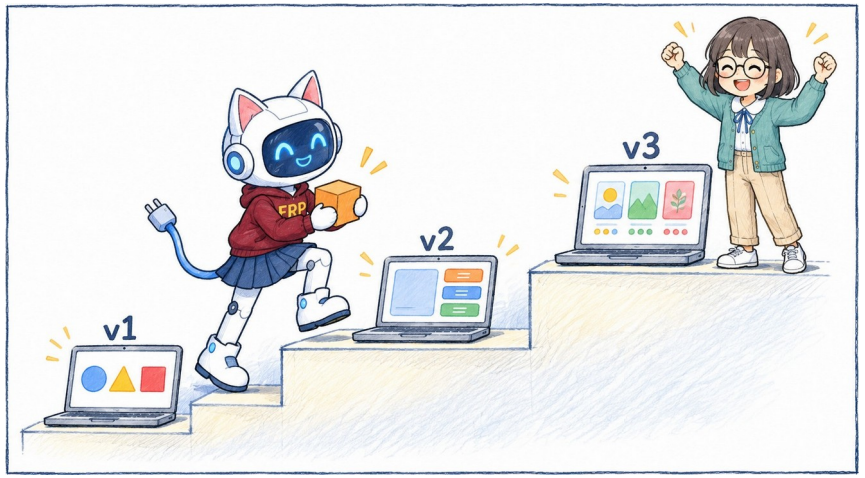
in your workspace

/plan

Type: "Write a short plan first. Do not write code yet."

Fixing a plan takes 1 minute. Fixing code takes 10.

Small steps



in your workspace

/build

Type: “Make the simplest working version first. Then we add one feature at a time.”

One big step can take 6 minutes, and you see nothing.

Tell it how to check



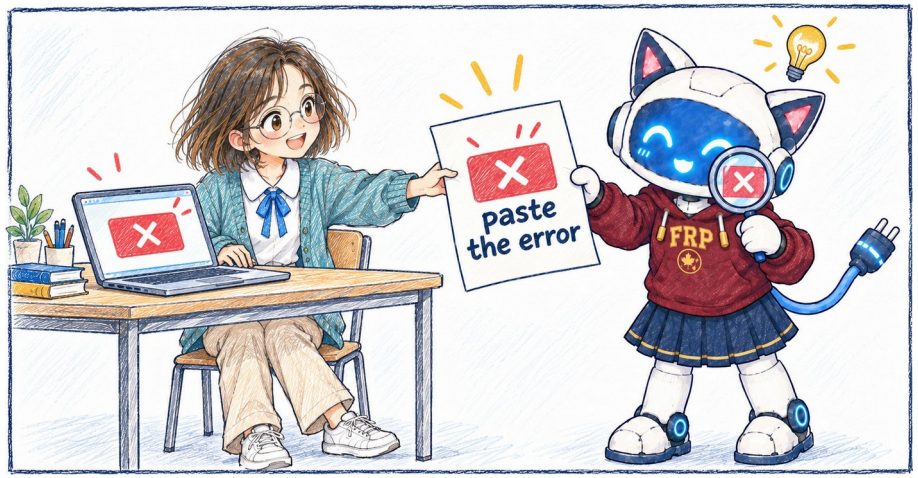
in your workspace

/check

Type: “When you finish, open the page, click like 3 times and tell me what changed.”

Our agents only checked that the code runs, and missed a bug you see in 2 seconds.

Paste the real error



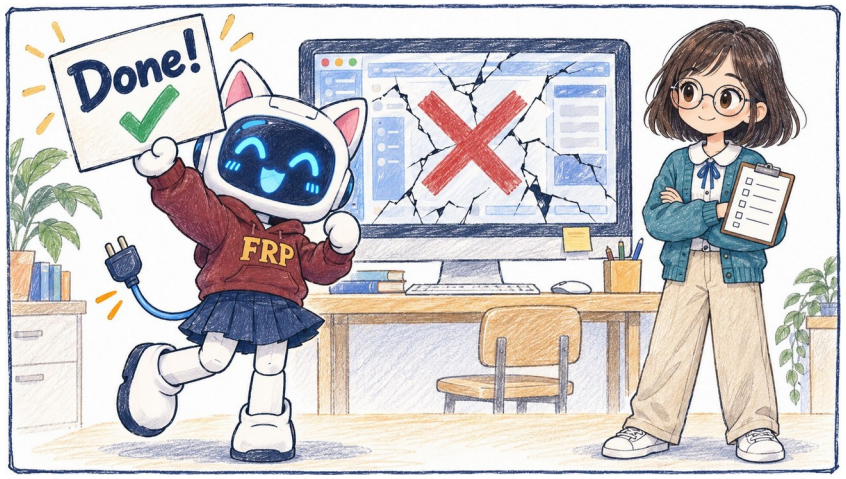
in your workspace

/fix

Type: **“Here is the error: (paste). Explain why it happens, then fix it.”**

> **“It does not work” gives the agent nothing. The error message gives it the answer.**

“Done” is a claim



in your workspace

/check

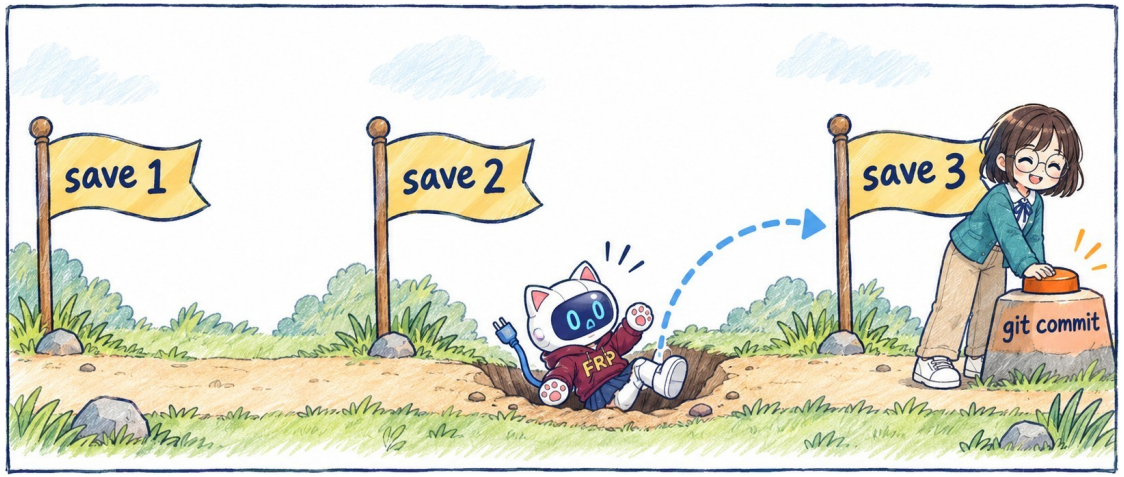
Type: “Show me how you know it works.”

> **In our tests the agent said “copied exactly” and “the server is running”. Both were false.**

Save after every step

NEW WORD

commit = a save point



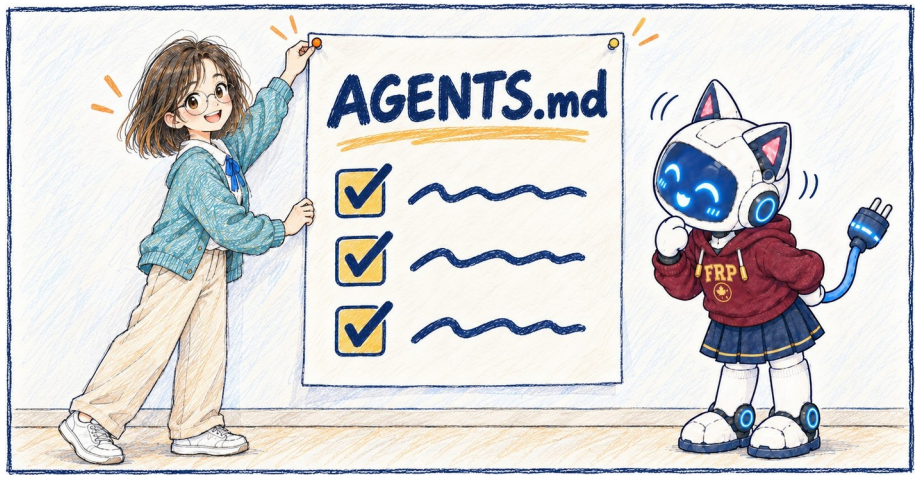
in your workspace

/save

Type: “Commit with git now, with a short message.”

If the agent breaks something, you go back to the last save.

Give it rules: AGENTS.md



in your workspace

AGENTS.md

Type: AGENTS.md: “Simple English. One HTML file. Never delete files without asking.”

> **The agent reads AGENTS.md every time. Write your rules once.**

8 sentences that make the agent better

BEFORE YOU BUILD

1 “Ask me up to 5 questions first.”

/ask

2 “Write a short plan. No code yet.”

/plan

8 AGENTS.md: your rules, written once

AGENTS.md

DURING

3 “The simplest working version first.”

/build

5 “Here is the error: … Explain, then fix.”

/fix

AFTER

4 “Open the page. What do you see?”

/check

6 “Show me how you know it works.”

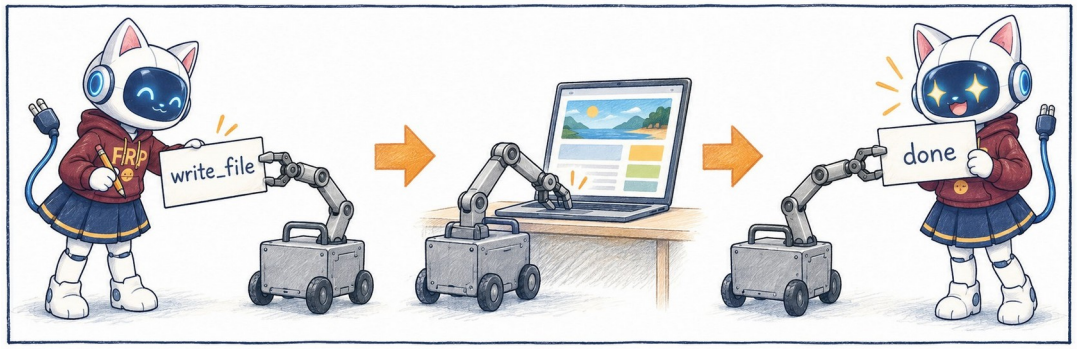
/check

7 “Commit with git now.”

/save

Remember 3: ask first, small steps, check it yourself.

Why OpenCode? One agent, 4 kinds of help



1 Tools

write files, run commands, fetch a web page

2 Skills

recipes for jobs we tested for you

3 Rules

AGENTS.md: read every time

4 Commands

/ask /plan /build /check /fix /save

Free, on our own model, and it works like the tools professionals use.

Skills: recipes the agent opens when it needs them

```
---
name: web-search
description: Find facts on the internet
  and read web pages. Use it for any
  question about the world.
---
# Search the web and read pages
1. Search: the websearch tool.
2. Read the best page: webfetch it.
3. Answer with the source URL.
   Never make up a number.
```

materials/skills/web-search/SKILL.md (shortened)

YOUR 8 SKILLS

web-search find facts, read pages

page-check test your page in a browser

image-gen make pictures

embeddings find similar things by meaning

face-watch face, eyes, blinks from a webcam

text-model train a tiny text model

browser-model train a model, run it in a page

grpo-train teach a model with rewards

Find them: materials/skills/ + SKILLS.md

It sees 1 line per skill. Task matches? It opens the recipe.

Fetch a URL: the agent reads a web page

```
> Fetch https://en.wikipedia.org/wiki/Astana
  When was it named Nur-Sultan, and when
  did it become Astana again? Source?

% WebFetch https://en.wikipedia.org/wiki/Astana

Renamed Nur-Sultan in March 2019;
reverted to Astana in September 2022.
Source: en.wikipedia.org/wiki/Astana
```

Real run in our class workspace, 24 Oct 2026

Use it for

the docs of an API

a dataset page on Hugging Face

an official page (a university, a bank)

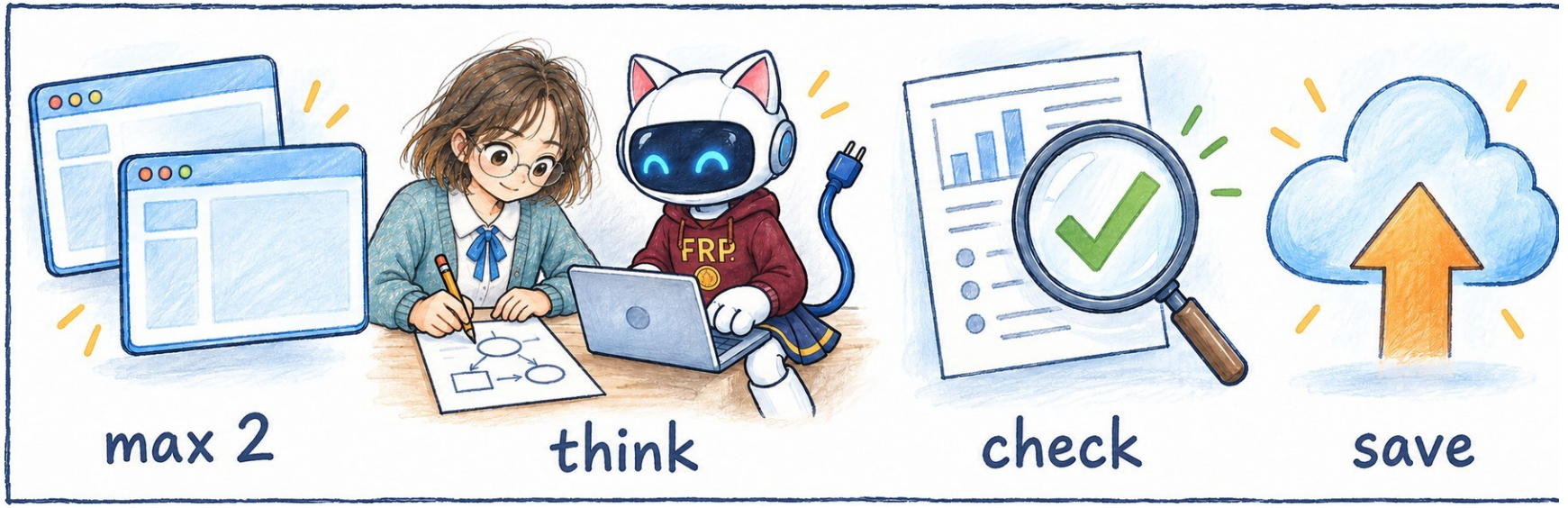
Does not work

pages with a login or a captcha

Search finds pages. Fetch reads one page.

> **Fetch reads today's page instead of guessing. You check the source.**

How we work with agents this week



Max 2 agent tabs
we share one server

Plan while it works

Check every result

Save on GitHub

Do not wait for the agent. Think while it works.

WORKSHOP, 3 MINUTES

Open your week plan



1 Open your workspace (same link as yesterday)

2 Open **projects/<your project>/WEEK-PLAN.md**

3 OpenCode: the terminal at the bottom

No terminal? Ctrl + ` then type opencode

4 Find task 1 and its box **Type this in OpenCode**

Your plan uses the tricks you just learned, step by step.

WORKSHOP, 20 MINUTES

Start task 1 of your plan

1 **paste the task 1 text**
it starts with /plan

2 **read the plan**
Goal? Input? Check? Limits? OK

3 **/build**
open the link it gives you

4 **/check**
click 3 buttons: do they work?

5 **/fix what you saw**
explain, then fix

6 **/save**
after each working step

Stuck? 1. /fix + what you see 2. your partner team 3. hand up

Answer in Kazakh, Russian or English. The agent understands all three.

Done = the “Done when” line of task 1. Then /save.

Your Demo Day sentence



In your plan, at the top

On Demo Day you say: "… X of N …"

What will you count? _____

Which day is your test? _____

Example: "On 100 real messages, our detector caught X of 50 scams and gave Y false alarms on 50 normal messages." Test day: Day 5.

A user, a goal, and a number that shows it works.

After lunch

13:00 Your Day 2 tasks

Your week plan, task by task

Stuck? /fix, then your partner team, then a mentor

Help your neighbour

Before you go

Show a mentor your “Done when”

/save

One line: what works, what is next

3 THINGS TO REMEMBER

1. A model guesses the next piece of text.
2. An agent is a model that uses tools in a loop. You give a clear task and check the result.
3. A good project has a user, a goal and a way to measure it.

Sources

Measured by us on the ISSAI FRP server, Qwen3.8-27B, 12 Oct 2026: tokens, next-word chances, the first Astana photo answers.

Measured by us, DeepSeek V4.1 Flash (our class model), 24 Oct 2026: 763 B weights counted from its files, 131,072 tokens in OpenCode, the Astana photo 4 of 4.

Llama 3.1 model card, Meta, Jul 2024: 15 trillion tokens; 30.84 million H100 GPU hours for 405B.

Kalai et al., “Why Language Models Hallucinate”, arXiv 2509.04664, Sep 2025.

Anthropic, “Building effective agents”, Dec 2024. Yao et al., ReAct, arXiv 2210.03629, 2022.

ISSAI KAZ-LLM (issai.nu.edu.kz/kazllm), Dec 2024. 20-sentence token test: tools/tokens_20_result.txt (12 Oct 2026).

TikTok newsroom, 18 Jun 2020. Meta, “How AI ranks content”, 29 Jun 2023. Big-model photo test: Codex CLI, 12 Oct 2026.

Astana renamed Nur-Sultan (Mar 2019) and back (Sep 2022): Wikipedia, “Astana”.

Photo: Dauren Nabijan, “Astana Esil view”, Wikimedia Commons, CC0 (public domain).